

UNITY EDITOR TOOL

Procedural Tree Generator

คู่มือการใช้งาน — เครื่องมือสร้างต้นไม้ Procedural สำหรับ HDRP
พร้อมระบบ LOD, Seed สุ่มแยกส่วน และระบบลมที่ขับเคลื่อนจาก WindZone จริงของ Unity

เวอร์ชัน	1.3.5 (20 กรกฎาคม 2026)
สภาพแวดล้อม	Unity 6000.3+ / HDRP 17.x
ตำแหน่งโค้ด	Assets/Scripts/Tool/TreeGenerator/ (namespace TreeTool)
โปรเจกต์	Yantra Project

สารบัญ

1. ภาพรวม
2. เริ่มต้นใช้งาน
3. ระบบ Seed (การสุ่ม)
4. โหมด Geometry: Procedural กับ Prefabs
5. พารามิเตอร์ทั้งหมด
6. ระบบ LOD
7. Material
8. ระบบลม: ใช้งานได้จริงกับ WindZone ของ Unity
9. การ Export เป็น Mesh Asset / ทำ Prefab
10. ข้อจำกัดและ Tips

ใหม่ในเวอร์ชัน 1.3

- **ระบบลมใช้งานได้จริง:** component ใหม่ `Tree Wind Zone Driver` อ่านค่าจาก `WindZone` ตัวจริงของ Unity ในฉาก (ทิศทาง/ความแรง/turbulence/pulse) แล้วส่งเป็น global shader property ให้ material ทุกอันอ่านได้ทันที ไม่ต้องตั้งค่าลมแยกทีละ material
- **ปรับลมแยกทีละส่วนตอนสร้างต้นไม้:** เพิ่ม `Wind Response` ที่ Trunk และที่แต่ละ Branch Level, เพิ่ม `Wind Flutter Response` ที่ Leaves — กำหนดได้เลยว่าส่วนไหนพลิ้วกับลมแค่ไหน ค่าถูก bake ลง vertex color โดยอัตโนมัติ
- เพิ่มไฟล์ `TreeWind.hlsl` — ฟังก์ชันคำนวณการขยับ vertex ตามลม พร้อมต่อเป็น Custom Function Node ใน HDRP Shader Graph ได้ทันที (ดูข้อ 8)
- Inspector เตือนอัตโนมัติเมื่อยังไม่มี Wind Zone ในฉาก พร้อมปุ่มเพิ่มให้ในคลิกเดียว

ใหม่ในเวอร์ชัน 1.2

- **รอยต่อกิ่งเนียนขึ้น:** กิ่งลูกโด้งออกจากทศกิ่งแม่แบบสมูท (Joint Smoothing) และโคนกิ่งบานกลืนเข้ากิ่งแม่ (Joint Flare) — จุดต่อสังเกตยากขึ้นมาก
- **Thickness Scale ต่อชั้นกิ่ง:** ปรับความหนาเฉพาะชั้นเดียว โดยส่งต่อให้ชั้นลูกตามลำดับชั้น (ลูกวัดสัดส่วนจากความหนาจริงของแม่) — ไม่ใช้การ scale ทั้งต้น
- **เหลื่อมแยกต่อส่วน:** Radial Segments ของลำต้น และของกิ่งแต่ละชั้น ปรับแยกอิสระ (ยกเว้นใบซึ่งเป็น card) — ย้ายออกจาก Mesh settings เดิม
- ปรับโค้ดทั้งหมดเป็น C# syntax สมัยใหม่เท่าที่ Unity 6 รองรับ (ประสิทธิภาพเท่าเดิม)

ใหม่ในเวอร์ชัน 1.1

- ขยายช่วงปรับค่าทั้งหมดให้กว้างขึ้นมาก (ความสูงถึง 100 ม., กิ่งหนากว่าแม่ได้, ใบถึง 300 ใบ/กิ่ง ขนาดถึง 10 ม.)
- ใบ: ปรับระยะห่างจากกิ่ง (Surface Offset เป็นช่วงสุม -2 ถึง +5 ม.) + Rotation Offset (องศา XYZ)
- จุดปักใบใหม่: มุมล่างซ้ายของ texture (โคนใบ) ปักกับกิ่งเสมอ และแนวแยง card ชี้ออกจากกิ่ง
- ลากปรับระยะที่ LODGroup โดยตรง ค่าจะ sync กลับเข้า settings ของ tool อัตโนมัติ
- Inspector ซ่อน field ที่ไม่เกี่ยวกับโหมด Geometry ที่เลือก
- ปรับค่าสลับขึ้นมาก: ระหว่างลาก slider จะ rebuild เฉพาะ LOD0 แบบเบา แล้ว rebuild เต็มอัตโนมัติเมื่อหยุดลาก

แก้ไข v1.3.1: ตัวอย่าง node wiring ของ Custom Function เดิมทำให้เกิด error `undeclared identifier` ถ้าใส่ `_float` ต่อท้ายใน Name field เอง (Shader Graph เดิมให้อยู่แล้ว) และมีจุดเสี่ยงต่อ Vertex Color เข้า Add ผิดที่ (ต้องเป็น Position ไม่ใช่ Vertex Color) — แก้ไขขั้นตอนในข้อ 8 ให้ถูกต้องแล้ว

ใหม่ใน v1.3.2 – 1.3.4: ข้อ 8 เพิ่มขั้นตอน Fragment stage แบบเต็ม (Base/Normal/Metallic/AO/Smoothness) พร้อม dropdown สลับรูปแบบ texture ระหว่าง **HDRP (Mask Map)** กับ **URP (Metallic/Occlusion แยกไฟล์)** เพิ่ม **Detail Map, Height Map** (vertex displacement), **Emission** ให้ครบแบบ HDRP/Lit ปกติ พร้อมพารามิเตอร์ปรับค่าแต่ละ map ได้แบบ HDRP Material จริง — `Metallic Scale`, `Smoothness/AO Remapping` (min-max), `Emission Intensity` และตาราง **ค่าเริ่มต้นแนะนำ** ครบทุก property

แก้ไข v1.3.5: ชื่อ Reference ในตารางข้อ 8 ปรับให้ตรงกับที่ใช้จริงใน `Tree-HDRP-Lit-PBR.shadergraph` (ไม่ใช่ชื่อที่เสนอไว้ตอนแรกใน v1.3.2-1.3.4) — `_AmbientOcclusionMap` (แทน `_OcclusionMap`), `_SmoothnessMinScale` / `_SmoothnessMaxScale` (แทน `_SmoothnessRemapMin/Max`), `_AmbientOcclusionMinScale` / `_AmbientOcclusionMaxScale` (แทน `_AORemapMin/Max`), และ Keyword Reference `MAPFORMAT` (ไม่มี underscore นำหน้า แทน `_MapFormat`) อัปเดต `Editor/TreeWindShaderGUI.cs` ให้ค้นหาด้วยชื่อจริงเหล่านี้แล้ว — เลือกแก้ script ให้ตรงกับกราฟที่มีอยู่ แทนที่จะให้แก้กราฟ เพราะกราฟ wiring ถูกต้องอยู่แล้ว

1 ภาพรวม

เครื่องมือสร้างต้นไม้แบบ procedural ภายใน Unity แนวคิดเดียวกับ Tree Creator แต่ออกแบบสำหรับ **HDRP**:

- ปรับค่าใน Inspector แล้วโมเดลอัปเดตทันทีใน Edit Mode (ลาก slider แบบ real-time)
- ค่าเกือบทั้งหมดเป็นช่วงสุม (min-max slider ลากได้)
- Seed แยก 3 ตัว — สุ่มทรงต้น / กิ่ง / ใบ แยกอิสระ
- LOD หลายระดับ + LODGroup อัตโนมัติ พร้อม cross-fade
- 2 โหมด: สร้าง geometry เอง หรือใช้ prefab (FBX) ที่ปั้นเอง — ใส่หลาย variant สุ่มเลือกได้
- **ระบบลมใช้งานได้จริง** ชับเคลื่อนจาก WindZone ตัวจริงของ Unity ปรับได้ทีละส่วนว่าลำต้น/กิ่ง/ใบ พริ้วแค่ไหน

ไฟล์	หน้าที่
<code>ProceduralTree.cs</code>	Component หลัก วางบน GameObject
<code>ProceduralTreeSettings.cs</code>	พารามิเตอร์ทั้งหมด
<code>TreeSkeleton.cs</code>	สุมโครงต้นไม้ (spline ลำต้น/กิ่ง + ตำแหน่งใบ)
<code>TreeMeshBuilder.cs</code>	แปลงโครงเป็น mesh ต่อ LOD (bake ข้อมูลลง vertex color)
<code>TreeWindZoneDriver.cs</code>	อ่าน WindZone จริงในฉาก ส่งเป็น global shader property
<code>TreeWind.hlsl</code>	ฟังก์ชันคำนวณการขยับ vertex ตามลม สำหรับ Shader Graph
<code>Editor/ProceduralTreeEditor.cs</code>	Inspector, ปุ่มสุม seed, stats, export, ปุ่มเพิ่ม Wind Zone
<code>Editor/TreeWindShaderGUI.cs</code>	Material Inspector ของ wind shader — dropdown สลับ HDRP/URP texture format
<code>Editor/MinMaxRangeDrawer.cs</code>	Slider แบบ min-max

2 เริ่มต้นใช้งาน

- คลิกขวาใน **Hierarchy** → **3D Object** → **Procedural Tree (Tool)** — ต้นไม้ถูก generate ทันที เป็นลูก `LOD0/LOD1/LOD2` + `LODGroup`
- เลือกตัวแม่ (component `ProceduralTree`) แล้วปรับค่า — โมเดลอัปเดตทันที ระหว่างลาก slider ระบบ rebuild เฉพาะ LOD0 แบบเบาให้ลื่น (~หลาย ms ต่อครั้ง) แล้ว rebuild ครบทุก LOD + tangents อัปเดตใหม่หลังหยุดลาก ~0.35 วินาที
- ใส่ Material จริงในช่อง **Bark Material / Leaf Material** (ก่อนใส่เป็นสี placeholder)
- ปุ่ม **Rebuild Now** = บังคับ generate ใหม่, **Export Meshes To Assets** = freeze เป็นไฟล์ asset
- กล่อง stats ด้านล่างแสดงจำนวนกิ่ง และ verts / tris / ใบ ของแต่ละ LOD

3 ระบบ Seed (การสุ่ม)

ทุกการสุ่มเป็น **deterministic** — settings เดิม + seed เดิม ได้ต้นเดิมเป๊ะ 100%

Seed	ควบคุม	ปุ่ม
Seed	ทรงลำต้นและทุกอย่างที่ต่อยอดจากมัน (seed หลักทั้งต้น)	New Tree
Branch Seed	เฉพาะกิ่ง — สลับตำแหน่ง/มุม/ความยาว/variant กิ่งใหม่ โดยลำต้นไม่เปลี่ยน	Shuffle
Leaf Seed	เฉพาะใบ — สลับตำแหน่ง/ขนาด/การหมุน/variant ใบใหม่ โดยกิ่งไม่เปลี่ยน	Shuffle

Workflow แนะนำ: กด New Tree จนได้โครงที่ชอบ → Shuffle กิ่งจนพุ่มสวย → Shuffle ใบเก็บรายละเอียด

4 โหมด Geometry: Procedural กับ Prefabs

Section **Geometry** เลือก source แยก 3 ส่วน: Trunk / Branch / Leaf (ผสมได้)

Inspector ซ่อน **field** ที่ไม่เกี่ยวข้องกับมอดิ: โหมด Procedural ซ่อนช่องใส่ prefab / โหมด Prefabs ซ่อน field ฝั่ง procedural (Leaf Shape, Radial Segments, Bark UV Tiling)

โหมด Procedural (ค่าเริ่มต้น)

ลำต้น/กิ่งเป็นท่อนตาม spline, ใบเป็น card (Quad / Cross / TripleCross) ใช้ texture + alpha clip

โหมด Prefabs (ใช้ FBX ที่ป้อนเอง)

ใส่ prefab ได้หลายอันต่อช่อง สุ่มเลือก variant ต่อลำต้น / ต่อกิ่ง / ต่อใบ (ผูก seed)

ส่วน	แกนการป้อน	Pivot
ลำต้น / กิ่ง	ยืดตามแกน +Y (ควรมี segment ตามแนวยาวพอให้ตัดโค้งสวย)	ที่โคน
ใบ / ช่อใบ	ยื่นตามแกน +Z (+Y = ด้านหน้าใบ) ขนาด ~1 unit	จุดเสียบกิ่ง

- Mesh ลำต้น/กิ่งถูก **ตัดโค้งตาม spline** และสเกลหน้าตัดตามรัศมีกิ่ง (taper/root flare มีผล)
- ทุกอย่าง bake รวมเป็น mesh เดียวต่อ LOD — ใช้ **Material ของ tree ไม่ใช่ของ prefab**

5 พารามิเตอร์ทั้งหมด

Trunk (ลำต้น)

ค่า	ความหมาย
Height	ความสูง (เมตร) — ช่วงสุ่ม สูงสุด 100 ม.
Radius	รัศมีโคนต้น — ช่วงสุ่ม สูงสุด 10 ม.
Taper	ความเรียวปลาย (0 = ทรงกระบอก, 1 = แหลม)
Root Flare / Height	โคนบานพิเศษ + ระยะที่บานขึ้นไป
Segments	จำนวนท่อนตามความสูง (มาก = โค้งเนียน)
Radial Segments v1.2	เหลี่ยมรอบลำต้น — เฉพาะลำต้น (สูงสุด 64) แยกจากกิ่งแต่ละชั้น
Crookedness	ความคดเคี้ยว
Lean	องศาเอียงทั้งต้น — ช่วงสุ่ม
Wind Response v1.3	ลำต้นพริ้วตามลมแค่ไหน (0 = แข็งนิ่ง, 1 = ปกติ) — default ต่ำ (0.25)

Branch Levels (ชั้นกิ่ง — list เพิ่ม/ลดชั้นได้)

ชั้น 0 งอกจากลำต้น, ชั้น 1 งอกจากชั้น 0 ต่อไปเรื่อย ๆ (default: Main Branches → Twigs)

ค่า	ความหมาย
Count	จำนวนกิ่งต่อกิ่งแม่ — ช่วงสุ่ม สูงสุด 100
Spawn Range	ตำแหน่งบนกิ่งแม่ที่งอกได้ (0 = โคน, 1 = ปลาย)
Angle	มุมทางออกจากกิ่งแม่ (องศา) — ช่วงสุ่ม
Azimuth Randomness	ความสุ่มการหมุนรอบกิ่งแม่ (0 = เกลียว golden angle)
Length Ratio	ความยาวเทียบกิ่งแม่ — ช่วงสุ่ม สูงสุด 3 เท่า
Length Falloff	กิ่งใกล้ปลายแม่สั้นลงเท่านี้
Radius Ratio	ความหนาเทียบกิ่งแม่ — เกิน 1 ได้ (หนากว่าแม่) สูงสุด 2
Thickness Scale v1.2	ตัวคูณความหนาเฉพาะชั้นนี้ — ปรับชั้นเดียว ไม่กระทบชั้นบน แต่ส่งต่อให้ชั้นลูกตามลำดับชั้น
Radial Segments v1.2	เหลี่ยมของกิ่งชั้นนี้ (แยกอิสระจากลำต้นและชั้นอื่น)
Joint Smoothing v1.2	สัดส่วนช่วงต้นกิ่งที่โค้งออกจากทิศกิ่งแม่แบบสมูท — ซ่อนรอยต่อ
Joint Flare v1.2	โคนกิ่งหนาขึ้นแล้วค่อยเรียว — กลืนโคนกิ่งเข้ากิ่งแม่
Taper / Gravity / Crookedness / Segments	เรียวปลาย / ห้อยลง(+)-เชิดขึ้น(-) / ความคด / จำนวนท่อน (สูงสุด 32)
Wind Response v1.3	กิ่งชั้นนี้ฟุ้งตามลมแค่ไหน — default กิ่งใหญ่ 1.0, กิ่งฝอย 1.8

Leaves (ใบ)

ค่า	ความหมาย
Shape	Quad / Cross / TripleCross (แสดงเฉพาะโหมด Procedural)
Count Per Branch	จำนวนใบต่อกิ่ง — ช่วงสุ่ม สูงสุด 300
Size	ขนาดใบ (เมตร) — ช่วงสุ่ม สูงสุด 10 ม.
Spawn Range	ตำแหน่งบนกิ่งที่ใบเกิดได้
Orientation Randomness	0 = เรียงตามกิ่ง, 1 = สุ่มเต็มที่
Surface Offset v1.1	ระยะห่างใบจากผิวกิ่ง — ช่วงสุ่ม -2 ถึง +5 ม. (ติดลบ = จมเข้ากิ่ง)
Rotation Offset v1.1	หมุนใบเพิ่มทุกใบ (องศา XYZ) — จุดหมุนคือโคนใบที่ปักกิ่ง
Wind Flutter Response v1.3	ใบสั่น/บิดเองในลมแค่ไหน (แยกจากการแกว่งของกิ่ง) — default 1.5
Min Branch Level	ใบเกิดบนกิ่งชั้นนี้ขึ้นไป (ลำต้น = 0)

ทิศทางใบ (v1.1): จุดปักของ card คือ มุมล่างซ้ายของ **texture (UV 0,0)** = โคน/ก้านใบ ปักติดกิ่งเสมอ และ card ถูกหมุนให้ **แนวทแยงชี้ออกจากกิ่ง** — วาด texture ใบแบบโคนอยู่มุมล่างซ้าย ปลายใบมุมบนขวา จะได้ทิศถูกต้อง อัตโนมัติ

Mesh

ค่า	ความหมาย
Bark UV Tiling	ความถี่ UV แนวตั้งของเปลือกต่อเมตร
Generate Tangents	เปิดไว้ถ้าใช้ normal map (ข้ามชั่วคราวระหว่างลาก slider เพื่อความสิ้น)

6 ระบบ LOD

ค่า	ความหมาย
Generate LOD Group	ปิด = สร้าง LOD0 เต็มรายละเอียดอันเดียว
Cross Fade	เฟดนุ่มตอนสลับ LOD
Compensate Leaf Size	LOD ไกลใบเหลือน้อย → ขยายใบที่เหลือชดเชย
Screen Height (ต่อระดับ)	LOD แสดงจนต้นเล็กกว่าสัดส่วนจอ — ระดับสุดท้าย = จุด cull
Radial Resolution (ต่อระดับ)	ตัวคูณเหลี่ยม (ลด poly ไกล ๆ)
Max Branch Level (ต่อระดับ)	ตัด geometry กิ่งชั้นลึก (ใบยังอยู่รักษา silhouette)
Leaf Density (ต่อระดับ)	สัดส่วนใบที่เหลือ (สุ่มตัดแบบ deterministic)

ค่า default: 3 ระดับ (0.45 / 0.18 / 0.05)

Sync สองทาง (v1.1): ลากปรับระบบน **LODGroup component** โดยตรง ค่าจะถูกดึงกลับเข้า settings ของ tool อัตโนมัติ — rebuild ครั้งถัดไปไม่เขียนทับค่าที่ปรับไว้

7 Material

- **Bark Material** — เลือกไม้ HDRP/Lit ปกติ (มี UV + tangent รองรับ normal map)
- **Leaf Material** — HDRP/Lit เปิด **Double-Sided** + **Alpha Clipping** ใส่ texture ใบ
- ปลอยว่าง = placeholder อัตโนมัติ (น้ำตาล/เขียว) — ใช้ชั่วคราวเท่านั้น
- Mesh มี 2 submesh เสมอ: 0 = เปลือก, 1 = ใบ

8 ระบบลม: ใช้งานได้จริงกับ WindZone ของ Unity

ต้นไม้จาก tool นี้ ใช้งานลมได้จริงแล้ว ขับเคลื่อนจาก **component** `WindZone` ตัวจริงของ **Unity** (อันเดียวกับที่ลาก Direction / Wind Main / Turbulence / Pulse ในฉาก) — ไม่ต้องตั้งค่าลมแยกทีละ material

หมายเหตุความถูกต้อง: ระบบลมภายในของ Unity ที่ใช้กับ SpeedTree เป็น pipeline ปิด (proprietary) ที่ bake ข้อมูล vertex เฉพาะรูปแบบของตัวเอง mesh ที่ tool นี้สร้างเองจึงต่อเข้า pipeline นั้นตรง ๆ ไม่ได้ — สิ่งที่ทำได้คืออ่านค่าจริงจาก **component** `WindZone` (ทิศทาง, ความแรง, turbulence, pulse) แล้วคำนวณการขยับ vertex ด้วยสูตรของเราเอง ผลคือ artist ยังใช้ workflow เดิม (ลาก WindZone ลงฉาก ปรับที่นั่นทีเดียว) ต้นไม้ทุกต้นในฉากขยับตาม

วิธีเปิดใช้งาน (3 ขั้นตอน)

- ในฉากต้องมี `Tree Wind Zone Driver` (Add Component → Tools/Tree Wind Zone Driver) — หรือกดปุ่ม “Add Wind Zone To Scene” ที่ขึ้นเตือนใต้ section Wind ใน `ProceduralTree` inspector เมื่อยังไม่มี (สร้าง `WindZone` + driver ให้พร้อมกันคลิกเดียว)
- ปรับค่าลมที่ **component** `WindZone` ตามปกติ (Mode, Wind Main, Turbulence, Pulse Magnitude/Frequency) — เหมือนตั้งค่าลมให้ต้นไม้ปกติของ Unity ทุกประการ
- ทำ material เปลือก/ใบด้วย **HDRP Shader Graph** ที่มี node **Custom Function** ชี้ไปที่ไฟล์ `TreeWind.hlsl` ฟังก์ชัน `TreeWindDisplacement_float` แล้วบวกผลลัพธ์เข้ากับตำแหน่ง vertex (ดูรายละเอียด node ด้านล่าง) — ใส่ material นี้ในช่อง Bark/Leaf Material ของต้นไม้

Component `Tree Wind Zone Driver` จะหา `WindZone` ที่ enable อยู่ตัวแรกในฉากให้อัตโนมัติ (หรือ assign เเจาะจงเองในช่อง `Wind Zone` ก็ได้) แล้วส่งค่าออกเป็น **global shader property** ทุกเฟรม จึงไม่ต้อง assign อะไรเพิ่มที่ material แต่ละอัน

ข้อมูลที่ bake ไว้ใน vertex color (ให้อัตโนมัติ ไม่ต้องทำเอง)

เมื่อเปิด **Wind** → **Bake Wind Data** (default เปิด) ทุก vertex มีน้ำหนักลมใน vertex color โดยคุณด้วยค่า **Wind Response** ของส่วนนั้น ๆ ที่ตั้งไว้ตอนสร้างต้นไม้ (ข้อ 5) เรียบร้อยแล้ว:

Channel	ข้อมูล	มาจาก
R	Main bend: โค้งตามความสูง (Bend Exponent) × Wind Response ของส่วนนั้น	Trunk/Branch Level <code>Wind Response</code>
G	Local sway: 0 โคนกิ่ง → 1 ปลายกิ่ง × Wind Response ของกิ่งนั้น (ลำดับ = 0, ใบบรับค่าจากกิ่งที่เกาะ)	Branch Level <code>Wind Response</code>
B	Leaf flutter: ความสั่น/บิดของใบเอง (เปลือก = 0)	Leaves <code>Wind Flutter Response</code>
A	Random phase ต่อกิ่ง/ต่อใบ (0–1)	สุ่มอัตโนมัติ

นี่คือจุดที่ตอบคำถาม “set แต่ละส่วนว่าจะฟรั้มแค่ไหน” — ปรับ `Wind Response` ที่ Trunk / ที่แต่ละ Branch Level / `Wind Flutter Response` ที่ Leaves ตอนสร้างต้นไม้ไม่ได้เลย ไม่ต้องแก้ shader เช่น ลำต้นแทบไม่ขยับ (0.25) → กิ่งใหญ่ขยับปานกลาง (1.0) → กิ่งผอมขยับเยอะ (1.8) → ใบสั่นเร็วสุด (1.5)

Custom Function node (Shader Graph) — ต่อครั้งเดียวจบ

`TreeWind.hlsl` เขียนสูตรรวมทั้งหมดไว้ในฟังก์ชันเดียวแล้ว (main bend + local sway + leaf flutter + turbulence) ไม่ต้องต่อ node เองทีละเส้น:

1. ใน HDRP Lit Shader Graph เพิ่ม node **Custom Function**, ตั้ง Type = **File**, ชี้ไปที่ `TreeWind.hlsl`, ช่อง **Name** ใส่ `TreeWindDisplacement`

สำคัญ: ห้ามใส่ `_float` ต่อท้ายใน Name field — Shader Graph เติม `_float` / `_half` ให้เองอัตโนมัติตาม precision ของกราฟ ถ้าใส่เข้าจะกลายเป็นเรียกหาฟังก์ชันชื่อ `TreeWindDisplacement_float_float` ซึ่งไม่มีจริง แล้วขึ้น error `undeclared identifier` สีแดง

2. ตั้ง Input ของ node ตามลำดับนี้ (ชื่อ/ชนิดต้องตรง): `PositionWS` (Vector3), `VertexColor` (Vector4), `MainAmplitude` (Float), `BranchAmplitude` (Float), `LeafAmplitude` (Float), `MainSpeed` (Float), `BranchSpeed` (Float), `LeafSpeed` (Float) และ Output หนึ่งช่อง: `PositionOffsetWS` (Vector3)
3. เพิ่ม node **Position** ตั้ง Space = **World** → ต่อเข้า `PositionWS`, และ **Vertex Color** → ต่อเข้า `VertexColor`
4. Amplitude/Speed ให้ **Expose** เป็น **material property** แล้วตั้งค่าเริ่มต้นแนะนำ: `MainAmplitude` $\approx$ 0.15–0.3, `BranchAmplitude` $\approx$ 0.1–0.2, `LeafAmplitude` $\approx$ 0.05–0.1, `MainSpeed` $\approx$ 0.6, `BranchSpeed` $\approx$ 1.8, `LeafSpeed` $\approx$ 5 (ปรับตามสเกลต้นไม้ได้)
5. แปลงเฉพาะ **offset** กลับเป็น **Object Space** (อย่าแปลงตำแหน่งรวมทั้งก้อน เสียพลัง่ายกว่า): เพิ่ม node **Transform** ตั้ง Type = **Direction**, From = **World**, To = **Object** แล้วต่อ `PositionOffsetWS` เข้า node นี้
6. เพิ่ม node **Position** อีกตัว ตั้ง Space = **Object** แล้วใช้ **Add**: **A** = Position (Object) ตัวนี้, **B** = output จาก Transform ซีก่อนหน้า (ห้ามต่อ **Vertex Color** เข้า **Add** โดยตรง — เป็นข้อผิดพลาดที่พบบ่อยเพราะ node หน้าตาคล้าย Position จนสับสน แต่ Vertex Color คือน้ำหนักกลม ไม่ใช่ตำแหน่ง)
7. ต่อ `Add.Out(3)` เข้า **Vertex Position** ใน Master Stack

ทำ shader graph นี้ 2 อัน (bark, leaf — ของใบเปิด Double-Sided + Alpha Clipping) แล้วใส่ในช่อง Bark Material / Leaf Material ตามปกติ ต้นไม้ทุกต้นที่ใช้ material นี้จะปลิวตาม WindZone ทันที

Fragment stage: PBR เต็มรูปแบบ + สลับ Texture Format ได้ (HDRP / URP) v1.3.2–1.3.5

ขั้นตอนข้างบนต่อแค่ **Vertex stage (ลม)** เท่านั้น — Shader Graph เปล่า ๆ ไม่มีช่อง Base Color / Normal Map / Mask Map ให้เหมือน HDRP/Lit อัตโนมัติ ต้องต่อ node ฝั่ง **Fragment** เอง (คนละ stage กับ Vertex ที่ทำไปแล้ว ไม่กระทบกัน)

Material ยังคงเป็น **HDRP เต็มรูปแบบ** (render ด้วย HDRP Lit target) — แค่เลือกได้ว่า **texture** ที่ป้อนเข้ามาเป็นแบบ **HDRP (Mask Map)** หรือแบบ **URP (Metallic + Occlusion แยกไฟล์)** ผ่าน dropdown บน Inspector ของ material เอง เลือกโหมดไหน อีกโหมดจะซ่อนช่องใส่และไม่ถูกนำไปคำนวณเลย (ตัด branch ที่ไม่ได้ใช้ออกตอน compile shader ด้วย Shader Feature keyword ไม่ใช่แค่ซ่อน UI เนย ๆ)

1. เพิ่ม Property ใน Blackboard — ตั้งชื่อ Reference ให้ตรงตามตารางนี้เป๊ะ (ตัวพิมพ์เล็ก/ใหญ่มีผล) เพราะ script TreeWindShaderGUI.cs จะค้นหาด้วยชื่อนี้:

Reference	Type	ใช้ทำ
<code>_BaseColor</code>	Color	สีปรับโทนคุณกับ Base Map
<code>_BaseMap</code>	Texture2D	สี/ลายเปลือกไม้หรือใบ
<code>_NormalMap</code>	Texture2D	normal map — ตั้ง Mode = Bump ในกล่อง property
<code>_NormalScale</code>	Float	ความแรง normal map (default 1)
<code>_Tiling</code>	Vector2	default (1, 1)
<code>_MaskMap</code>	Texture2D	HDRP: Metallic(R) / AO(G) / Detail(B) / Smoothness(A)
<code>_MetallicGlossMap</code>	Texture2D	URP: Metallic(R) / Smoothness(A)
<code>_AmbientOcclusionMap</code>	Texture2D	URP: AO (อ่านช่อง G ตาม convention ของ URP/Standard)
<code>_MetallicScale</code>	Float	คุณกับค่า Metallic ที่อ่านได้ — ปรับความเป็นโลหะโดยไม่แก้ texture
<code>_SmoothnessMinScale/Max</code>	Float×2	remap Smoothness [0,1]→[Min,Max] — ตรงกับ Smoothness Remapping ของ HDRP Material จริง
<code>_AmbientOcclusionMinScale/Max</code>	Float×2	remap AO เหมือนกัน — ตรงกับ Ambient Occlusion Remapping ของ HDRP Material จริง

2. เพิ่ม Keyword แบบ Enum ใน Blackboard (+ → Keyword → Enum):

- Name = `Map Format` , **Reference** = `MAPFORMAT` (พิมพ์ตรง ๆ อย่าปล่อยให้ auto-gen)
- Entries: `HDRP` (index 0) แล้ว `URP` (index 1) — เรียงลำดับนี้เท่านั้น ต้องตรงกับ script
- Definition = **Shader Feature** (ไม่ใช่ Global/Multi Compile — คอมไพล์แยก variant ต่อ material จริง ๆ)

3. ต่อ node ฝั่ง Fragment:

UV(0) → Tiling And Offset (Tile = _Tiling) → ป้อนเข้า Sample Texture 2D ทุกตัวด้านล่าง

Sample Texture 2D (_BaseMap) → RGB × _BaseColor.rgb → Base Color
→ A → Alpha

Sample Texture 2D (_NormalMap, Type=Normal) → Normal Strength(_NormalScale) → Normal (Tangent Space)

Sample Texture 2D (_MaskMap).R ↴

Sample Texture 2D (_MetallicGlossMap).R ↴ Keyword node #1 (ลาก MAPFORMAT มาวาง)

ช่อง "HDRP" ← MaskMap.R ช่อง "URP" ← MetallicGlossMap.R → Out × _MetallicScale → Metallic

Sample Texture 2D (_MaskMap).G ↴

Sample Texture 2D (_AmbientOcclusionMap).G ↴ Keyword node #2 (ลาก MAPFORMAT มาอีกตัว)

ช่อง "HDRP" ← MaskMap.G ช่อง "URP" ← OcclusionMap.G

→ Out → Lerp(A=_AmbientOcclusionMinScale, B=_AmbientOcclusionMaxScale, T=Out) → Ambient Occlusion

Sample Texture 2D (_MaskMap).A ↴

Sample Texture 2D (_MetallicGlossMap).A ↴ Keyword node #3 (ลาก MAPFORMAT มาอีกตัว)

ช่อง "HDRP" ← MaskMap.A ช่อง "URP" ← MetallicGlossMap.A

→ Out → Lerp(A=_SmoothnessMinScale, B=_SmoothnessMaxScale, T=Out) → Smoothness

เพิ่มปรับค่าแต่ละ Map ได้แบบ HDRP Material จริง: `_MetallicScale` คูณตรง ๆ กับ Metallic ส่วน

Smoothness/AO ใช้ **Lerp node** (T = ค่าที่อ่านจาก map, A = ...RemapMin, B = ...RemapMax) — กลไกเดียวกับที่ Inspector ของ HDRP Material เรียกว่า "Smoothness Remapping" / "Ambient Occlusion Remapping" ทุกประการ ไม่ใช่แค่ตั้งชื่อให้คล้าย

หมายเหตุ: ลาก node **Keyword** จาก Blackboard (ไม่ใช่ Branch node) — Keyword node ที่มาจาก Enum keyword จะมีช่อง input ชื่อตรงกับ entry ที่ตั้งไว้ (HDRP / URP) ให้อัตโนมัติ ลาก `MAPFORMAT` มาวางในกราฟ 3 ครั้ง สำหรับ Metallic/AO/Smoothness (ได้ node แยกกัน 3 ตัว ใช้ keyword เดียวกัน)

จุดพลัดบอย: node Sample Texture 2D ของ `_NormalMap` ต้องตั้ง dropdown เป็น **Type = Normal** (ไม่ใช่ Default) ไม่งั้นสีปกติจะเพี้ยน

4. เฉพาะ material ใบ (Leaf): เปิด **Alpha Clipping + Double-Sided** ใน Graph Settings, ต่อ `_BaseMap.A` เข้า **Alpha** ของ Fragment ด้วย

5. Detail Map (ลายละเอียดซ้อนทับ เช่น รอยแตกเปลือกไม้ระยะใกล้ — convention เดียวกันทั้ง HDRP/URP ไม่ต้องมี dropdown แยก):

Reference	Type	ใช้ทำ
<code>_DetailMap</code>	Texture2D	R=Detail Albedo(overlay,0.5=ไม่มีผล) / G=Detail Normal Y / B=Detail Smoothness(overlay) / A=Detail Normal X
<code>_DetailMask</code>	Texture2D	grayscale, default ขาวล้วน = detail เต็มที่ทุกจุด
<code>_DetailTiling</code>	Vector2	tiling แยกของ detail (ปกติถือว่า <code>_Tiling</code> หลายเท่า)
<code>_DetailAlbedoStrength</code>	Float	0-2
<code>_DetailNormalStrength</code>	Float	0-2
<code>_DetailSmoothnessStrength</code>	Float	0-2

UV(0) → Tiling And Offset (Tile = `_DetailTiling`) → Sample Texture 2D (`_DetailMap`, Type=Default)
Sample Texture 2D (`_DetailMask`) → R → mask

DetailAlbedo = (`_DetailMap.r` - 0.5) × 2 × `_DetailAlbedoStrength` × mask
→ Add เข้ากับ Base Color ที่ต่อไว้ในข้อ 3 (ก่อนออก Fragment.Base Color)

DetailNormalTS = Normalize(float3(`_DetailMap.a`×2-1, `_DetailMap.g`×2-1, 1))
→ Normal Blend node (Base Normal จากข้อ 3, Detail Normal นี้, Strength=`_DetailNormalStrength`×mask)
→ Out → Normal (Tangent Space) (แทนที่ output เดิมจาก `_NormalMap` อย่างเดียว)

DetailSmoothness = (`_DetailMap.b` - 0.5) × 2 × `_DetailSmoothnessStrength` × mask
→ Add เข้ากับ Smoothness ที่ต่อไว้ในข้อ 3 ก่อนออก Fragment.Smoothness

6. Height Map (ขุน/ยุบผิวเปลือกแบบเบา ๆ ผ่านการขยับ vertex ตามแนว normal — ไม่ใช่ Pixel/Parallax Occlusion Mapping เพราะกึ่ง/ไม่มี vertex และ instance เยอะมาก POM ต่อพิกเซลจะแพงเกินจำเป็น สำหรับพีช):

Reference	Type	ใช้ทำ
<code>_HeightMap</code>	Texture2D	grayscale height (0 = ยุบ, 1 = ขุน)
<code>_HeightScale</code>	Float	ระยะยุบ/ขุนสูงสุด (เมตร)

ต่อใน **Vertex stage** (ต่อจากผลลัพธ์ Add ของข้อมูลที่ทำไว้แล้ว):

Sample Texture 2D LOD (`_HeightMap`, LOD = 0) → R → height ⚠ ต้องใช้ "Sample Texture 2D LOD"
ไม่ใช่ "Sample Texture 2D" ธรรมดา - vertex shader ไม่มี mip derivative ใช้ node ปกติไม่ได้

HeightOffset = (height - 0.5) × `_HeightScale`
Position (Object Space) → Normal (Object Space) × HeightOffset → Add ตัวใหม่:
A = ผลลัพธ์ Add เดิม (ตำแหน่งหลังใส่ลมแล้ว), B = Normal(Object)×HeightOffset
→ Out → ต่อเข้า Vertex Position แทนที่ Add เดิม (Add ใหม่นี้เป็นตัวสุดท้ายก่อนเข้า Master Stack)

7. Emission (จุดเรืองแสง เช่น โบ๊ม่เรืองแสงเวทมนตร์/ยันต์ ถ้าไม่ใช่เว้น Emission Color ไว้ที่ดำ):

Reference	Type	ใช้ทำ
<code>_EmissionMap</code>	Texture2D	RGB emission (default ค่า = ไม่เรือง)
<code>_EmissionColor</code>	Color (HDR)	สี
<code>_EmissionIntensity</code>	Float (nits)	ตัวคูณความแรงแยกจากสี — วิธีเดียวกับที่ HDRP Material ใช้จริง

ต่อ: `Sample Texture 2D (_EmissionMap).RGB × _EmissionColor × _EmissionIntensity` → **Emission** ใน Fragment (อย่าลืมเปิด **Emission** ใน Graph Settings ถ้า Shader Graph ซ่อน slot นี้ไว้โดย default)

8. ค่าเริ่มต้นแนะนำทั้งหมด (Recommended Defaults)

ตั้งค่า default บน node/property ในกราฟให้ตรงตามนี้ตั้งแต่แรก จะได้ material ที่ดูสมเหตุสมผลทันที โดยไม่ต้องลองผิดลองถูก (ปรับต่อได้เสมอจาก material ที่ export ออกมา):

Property	ค่าเริ่มต้นแนะนำ	เหตุผล
<code>_BaseColor</code>	ขาวล้วน (1,1,1,1)	ให้ texture คุณเพิ่มเติมที่ ไม่มีสีทับ
<code>_NormalScale</code>	1	ความแรง normal ตามที่ป้อนมาเป๊ะ
<code>_Tiling</code>	(1, 1)	ไม่ทำซ้ำ ใช้ตาม UV ที่ป้อนมา
<code>_MetallicScale</code>	1 (เลือก/ใบ) หรือ 0.3–0.5 ถ้าอยากลดความมันแบบโลหะ	พืชแทบไม่เป็นโลหะ ปกติ Mask Map ควรมี Metallic ต่ำอยู่แล้ว
<code>_SmoothnessMinScale/Max</code>	0.0 / 0.6	เลือก/ใบไม่ควรมันวาวเกิน (ค่าเต็ม 1 จะดูเหมือนพลาสติก)
<code>_AmbientOcclusionMinScale/Max</code>	0.0 / 1.0	ใช้ค่า AO ตรงจาก texture เต็มช่วง ไม่บีบ
<code>_DetailAlbedoStrength</code>	0.5–1	ให้เห็น texture รายละเอียดแต่ไม่กลบสีหลัก
<code>_DetailNormalStrength</code>	0.5–1	นูนพอสังเกตเห็นได้ ไม่ล้นจน normal เพี้ยน
<code>_DetailSmoothnessStrength</code>	0.3	detail smoothness ปกติควรบางเบาว่า albedo/normal
<code>_DetailTiling</code>	(4,4) ถึง (8,8)	สูงกว่า <code>_Tiling</code> หลักหลายเท่า ให้เห็นเป็นรายละเอียดระยะใกล้
<code>_HeightScale</code>	0.01–0.02 (1–2 ซม.)	ผิวเลือกไม่นูนเบา ๆ ค่าสูงกว่านี้จะดูเป็นก้อนเกินจริง
<code>_EmissionColor</code>	ดำ (0,0,0)	ปิด emission ไว้ก่อน จนกว่าจะต้องใช้
<code>_EmissionIntensity</code>	0	คู่กับข้างบน — เปิดใช้ค่อยปรับขึ้น
MainAmplitude / BranchAmplitude / LeafAmplitude (ลม)	0.15–0.3 / 0.1–0.2 / 0.05–0.1	ดูหัวข้อ Custom Function ด้านบน

9. ตั้ง Custom Editor GUI ให้ได้ dropdown ที่ซ่อนช่องอัตโนมิติ: ที่ **Graph Settings** → **Custom Editor GUI** ใส่ `TreeTool.EditorTools.TreeWindShaderGUI` (จากไฟล์ใหม่ `Editor/TreeWindShaderGUI.cs`) — material จะมี

dropdown “**Format**” ที่ด้านบนสุด ของ Inspector เลือก **HDRP (Mask Map)** จะซ่อนช่อง Metallic/Occlusion ของ URP ไปเลย และกลับกัน

ข้อแลกเปลี่ยนที่ควรรู้: การตั้ง Custom Editor GUI เอง จะแทนที่ **Inspector** มาตรฐาน ของ **HDRP** ทั้งหมด (เสีย foldout อย่าง Surface Options/Advanced Options ของ HDRP ไป) `TreeWindShaderGUI` วาด field ที่จำเป็นให้ครบ (Base/Normal/Format dropdown/field อื่น ๆ ที่ expose ไว้ + Render Queue/GPU Instancing) แต่หน้าตาเรียบง่ายขึ้นกว่า HDRP Lit ปกติ ถ้าไม่ต้องการ dropdown ซ่อนช่อง เว้นช่อง Custom Editor GUI วางไว้ได้ — Map Format ยังสลับได้ปกติ (Unity สร้าง dropdown enum ให้อัตโนมัติจาก Blackboard keyword) แต่ทั้งสองชุด field จะโชว์พร้อมกันเฉย ๆ (ค่าที่ไม่ได้เลือกก็ยังไม่ถูกใช้เหมือนเดิม)

ข้อจำกัด

- Custom Function อ่านค่า global ที่ `Tree Wind Zone Driver` ตั้งไว้ — ถ้าไม่มี **driver** ในฉาก ต้นไม้จะนิ่ง (inspector ของ `ProceduralTree` จะเตือนพร้อมปุ่มเพิ่มให้อัตโนมัติ)
- ระหว่าง Edit Mode (ไม่ได้กด Play) การขยับจะอัปเดตเป็นช่วง ๆ ตามที่ editor repaint ไม่ liquid เท่า Play Mode — เข้า Play Mode เพื่อดูผลลัพท์ที่สุด
- ค่า Wind Response ที่ bake ไว้เป็นแค่ “น้ำหนัก” ไม่ใช่ระยะจริง (เมตร) — ระยะจริงคือ Amplitude บน material

9 การ Export เป็น Mesh Asset / ทำ Prefab

- Mesh ปกติไม่ถูก save ลง scene (ไฟล์เล็ก) และ regenerate เองตอนโหลด
- ตั้งชื่อ GameObject → กด **Export Meshes To Assets** → mesh ทุก LOD ถูกบันทึกที่ `Assets/GeneratedTrees/<ชื่อต้น>/` และ MeshFilter ชี้ไปที่ asset นั้นที่
- ลาก GameObject ลง Project เป็น prefab ได้เลย — แก่ค่าอีกเมื่อไหร่ค่อย export ทับใหม่

10 ข้อจำกัดและ Tips

- โหมต Prefabs: Radial Resolution ต่อ LOD ไม่มีผลกับ mesh จาก prefab (Max Branch Level / Leaf Density ยังช่วยลด poly)
- ยังไม่มี billboard LOD ระดับสุดท้าย (ใช้ cull แทน) และไม่สร้าง collider ให้
- ยังไม่มี Shader Graph asset สำเร็จรูปให้ลากใช้เลย ต้องต่อ Custom Function node เอง 1 ครั้งตามข้อ 8 (ไฟล์ `.shadergraph` เป็น JSON ที่เปราะบางมากถ้าสร้างโดยไม่ผ่าน Shader Graph editor)
- ต้นไม้อาจจำนวนมาก: ลด Count ของ Twigs แล้วเพิ่มใบต่อกิ่งแทน — พุ่มแน่นใน poly ถูกกว่า
- ป่าหลากหลาย: settings เดียวกัน เปลี่ยนแค่ Seed ต่อต้น
- Scene มีต้นไม้เยอะแล้วเปิดซ้ำ: export mesh เป็น asset จะไม่ต้อง regenerate ตอนโหลด